

HelixQA

HelixQA

Tagline: Anti-bluff QA orchestration — autonomous, cross-platform sessions where every PASS carries captured evidence that a real user can use the feature.

Summary: HelixQA is an anti-bluff QA orchestration framework for cross-platform testing (Android, Android TV, Web, Desktop) that combines YAML test banks, real-time crash detection, step-by-step evidence capture, and LLM-plus-computer-vision autonomous QA sessions to prove features genuinely work end-to-end. It is the Constitution's mandated QA test-type (§11.4.169).

Short description: An anti-bluff QA orchestrator (Go) that runs written test banks and fully-autonomous, LLM-and-vision-driven QA sessions across platforms — detecting crashes, validating each step against captured evidence (screenshots, logcat, video, stack traces), and auto-generating evidence-rich tickets for AI fix pipelines.

Long description: HelixQA is a Go framework whose single design centre is the Constitution's §11.4 Operative Rule: the bar for shipping is not "tests pass" but "users can use the feature," so every PASS it emits must carry positive runtime evidence captured during execution. It runs two complementary modes. First, **written test banks** — YAML suites of TC-xxx cases with platform targeting, priority, steps (name/action/expected), tags, and documentation refs — executed with per-step validation, real-time crash/ANR detection (ADB for Android, process monitoring for web/desktop), centralized evidence collection, and auto-generated Markdown tickets for AI fix pipelines. Second, a **fully-autonomous QA session** that uses LLM-powered agents and computer vision to navigate apps by itself in four phases: setup (select LLMs, build a feature map from project docs, spawn CLI agents, init the vision engine), doc-driven verification of every documented feature, curiosity-driven exploration of edge cases and undocumented behaviour, then report-and-cleanup into Markdown/HTML/JSON with video-timestamp-linked evidence. It integrates four external Go submodules (LLMsVerifier, LLMOrchestrator, VisionEngine, DocProcessor) and reuses the

challenges and containers infrastructure. Its own suite is held to the same bar via make anti-bluff (scan + behaviour-anchor manifest + mutation ratchet) and an 8-phase orchestrator Challenge with a built-in §1.1 mutation. A 15-row test-type coverage matrix ties every advertised capability to an executable asset and a captured-evidence shape.

Why we built it: Conventional QA green-lights on “the assertion passed,” which is exactly how the failure class the Constitution calls *bluffing* slips through — a feature reported working while it is broken for the real user. HelixQA was built to make that impossible for QA specifically: it refuses to score a PASS without physical evidence (screenshot, logcat, video, stack trace, report) captured under real execution, and it treats a green summary line with no such evidence as a critical defect equal to a missing feature. It also solves the labour problem — comprehensive manual QA across many platforms does not scale — by making the sessions fully autonomous.

Why it’s a game-changer: It fuses two things that are usually separate: rigorous, evidence-backed QA gating and autonomous, self-driving exploration. An LLM+vision agent can open the actual app, verify every documented feature, hunt for undocumented bugs, *and* produce a court-quality evidence trail — so “we tested it” becomes “here is the video, the logcat, and the ticket.” Because it is the Constitution-named QA submodule, adopting it upgrades QA honesty across every consuming product at once.

What’s innovative: - **Anti-bluff evidence contract** — every check’s PASS is bound to captured runtime evidence; CI-green is necessary, never sufficient. - **Autonomous doc-driven + curiosity-driven exploration** — verifies documented features *and* probes edge cases (empty inputs, rapid interactions, undocumented paths). - **Vision oracle** — GoCV mechanical vision plus LLM Vision API “sees” the running UI, not just token/property assertions. - **Structure-not-prose test banks** — bank strings describe structure and drive LLM-generated question prompts at runtime (CONST-046), avoiding hardcoded localized text. - **Tickets built for AI fix pipelines** — auto-generated Markdown issues with full evidence, ready to feed downstream repair agents.

How it is used across all products (the powers it gives): - **Mandatory QA test-type:** Constitution §11.4.169 names the `helix_qa` submodule as one of 13 required test types — “proper written test banks (suites) for every application/service/platform AND comprehensive fully-autonomous QA sessions,” each PASS citing captured physical evidence; there is no `--skip-helixqa` escape hatch. - **Autonomous QA sessions:** `helixqa autonomous --project ... --platforms android,desktop,web` drives real apps unattended to a coverage target, emitting reports, tickets, and videos. - **Test banks / suites:** ≥30 YAML banks (round-219 floor),

platform-targeted, priority-ranked, traceable to docs. - **Captured evidence:** screenshots, logcat, video, stack traces, timeline — centralized and linked from every report. - **Independent verdicts (§11.4.141 independence principle):** the Constitution's independent-review discipline (a reviewer/judge distinct from the producer) is mirrored in HelixQA's design — its LLM-powered issuedetector and vision oracle judge the running app's behaviour independently of the agent that navigated it, rather than the agent grading its own work. - **Gate + mutation ratchet:** make qa-all / make anti-bluff and challenges/scripts/helixqa_orchestrator_challenge.sh (8 phases, built-in §1.1 mutation) keep HelixQA's own honesty proven.

Biggest challenges + how solved: - *Preventing false positives in QA itself* → every step validated against captured evidence; PASS without evidence is a defect; behaviour-anchor manifest ties each advertised capability to a test (CONST-035). - *Driving heterogeneous platforms* → a navigator package with platform-specific ActionExecutors (ADB, Playwright, X11) and per-platform crash detectors (android/web/desktop). - *Making autonomous agents useful, not chaotic* → LLMsVerifier scores/selects models, LLMOrchestrator manages headless CLI agents (opencode, claude-code, gemini, junie, qwen-code), DocProcessor builds the feature map, VisionEngine grounds decisions in real pixels. - *Localization-safe banks* → banks describe structure; user-facing prompt text is LLM/resource-loaded at runtime (CONST-046), not hardcoded English. - *Proving the gates aren't shams* → paired §1.1 mutations (strip a type's evidence-capture or anti-bluff assertion → the gate must FAIL) and a mutation ratchet.

Tech/mechanism stack (why + how): - **Go 1.24+ orchestrator** — *why:* fast, portable, composable CLI; *how:* cmd/helixqa subcommands run / list / report / autonomous / version. - **YAML test banks (pkg/testbank)** — *why:* declarative, platform-filtered suites; *how:* version/name/test_cases[] with id, category, priority, platforms, steps[], documentation_refs[]. - **Crash/ANR detectors (pkg/detector)** — *why:* catch real failures live; *how:* ADB (pidof/logcat/screenshot) for Android, pgrep for web/desktop. - **Evidence collection (pkg/evidence, pkg/session)** — *why:* every PASS needs physical proof; *how:* screenshots/logcat/video/stack traces + SessionRecorder timeline. - **Autonomous session (pkg/autonomous, pkg/navigator, pkg/issuedetector)** — *why:* scalable, self-driving QA; *how:* 4-phase SessionCoordinator + ActionExecutors (ADB/Playwright/X11) + LLM bug detection across visual/UX/accessibility/functional. - **External submodules** — *why:* reuse, decoupling (CONST-051); *how:* LLMsVerifier (model scoring), LLMOrchestrator (headless CLI agents), VisionEngine (GoCV + LLM Vision), DocProcessor (feature-map/coverage). - **Anti-bluff gates + mutation ratchet** — *why:* the §1.1 covenant; *how:* make anti-bluff scan + behaviour-anchor manifest + mutation ratchet; helixqa_orchestrator_challenge.sh 8-phase validator. - **15-row**

coverage matrix (docs/test-coverage.md) — *why*: CONST-050(B) closed test-type set; *how*: each row bound to an executable asset and a captured-evidence shape.

Public links: - Canonical (public): <https://github.com/HelixDevelopment/helixqa> - Mirror (public): <https://github.com/vasic-digital/HelixQA> - Install: `go install digital.vasic.helixqa/cmd/helixqa@latest` · License: Apache-2.0

Suggested diagrams/illustrations (OpenDesign): 1.

Autonomous QA-session loop — the 4 phases (Setup → Doc-Driven Verification → Curiosity-Driven Exploration → Report & Cleanup) as a cycle, with evidence artefacts dropping out at each step. 2. **Anti-bluff evidence pipeline** — test step → detector + vision oracle → captured evidence (screenshot/logcat/video/stack trace) → PASS, with a “no evidence → critical defect” reject branch. 3. **Cross-platform navigator** — one orchestrator fanning to ADB (Android/TV), Playwright (Web), X11 (Desktop) action executors. 4. **Constitution → HelixQA governance** — §11.4.169’s 13 mandatory test types with `helix_qa` highlighted as the QA pillar, feeding the 15-row coverage matrix.

Site relevance: both

Priority tier: Helix-primary (governance pillar)

Source provenance: - /Volumes/T7/Projects/helixqa/README.md (status banner round 219, anti-bluff design centre, features, architecture packages, autonomous 4-phase session, external modules, test-bank conventions, governance pointers, make targets, license). - /Volumes/T7/Projects/helixqa/CONSTITUTION.md (inheritance from HelixConstitution, anti-bluff binding mandate, mocks-unit-only, §1.1 mutation pairing). - /Volumes/T7/Projects/constitution/Constitution.md §11.4.169 (lines 9787-9817, verbatim mandate naming `helix_qa` with written banks + autonomous sessions among 13 mandatory test types) and §11.4.141 (lines 9851-9877, independent-review principle: reviewer distinct from producer). - /Volumes/T7/Projects/vasic/_analysis/github-helix-others.md and github-vasic-digital.md (repo visibility = public; “AI-driven QA orchestration for multi-platform testing”; ecosystem role, consumes challenges/DocProcessor).